

Detecting Clickbait Titles

Christopher Lucinski

University of Guelph

Guelph, Canada

clucinsk@uoguelph.ca

Abstract— This document reports the findings of my final project for CIS*4020. It explores the topic of identifying clickbait titles through various machine learning models. The models used in this study returned high accuracy values of upwards of 90%. This report also discusses potential for moving forward beyond what is in this study.

Keywords— clickbait, natural language processing, word frequency, analysis

I. INTRODUCTION

In this current day and age, much of all media exists and is consumed online in a digital format. This is due to the rise of smartphones, portable computers, and the internet making it extremely easy to access seemingly infinite content from wherever a consumer may be. Part of this digital media revolution is the dying out of printed media. Print media is not only harder to access for an everyday consumer, but it also costs extra money and physical resources to print out and distribute. The advantage of digital media is that it doesn't cost paper or ink to create, and the potential audience is anyone with a digital device and an internet connection. This has led to a massive increase in the volume of media being created. But with the number of hours in a day staying the same, media creators have been forced to come up with creative ways to compete and attract the consumer to spend time on their own content. One method of enticing media consumers is infamously known as the "clickbait" title.

A clickbait title is a title or headline written with the goal of tricking or baiting a reader into clicking on a piece of media by being catchy and purposefully vague [1]. By not actually providing any solid information about the content, it piques the reader's interest enough that they click on the content to find out more about it. Clickbait titles are usually used for articles written by entertainment media outlets and internet videos like the ones found on YouTube. However, it is also not unheard of for news outlets to use clickbait titles for their articles in an attempt to

increase their online reader count as well. With this rise in titles that take advantage of readers' emotions and natural curiosity to give up their time, a question is raised: Is there a way for a program to methodically and automatically identify whether a title is clickbait?

While there appears to already be a bit of research on this particular topic, it's important to gain a basis of understanding regarding how to approach this problem. Since this a problem regarding the interpretation and identification of text, it is logical to assume that this is a problem that can be approached and potentially solved through methods of natural language processing.

A. Natural Language Processing

The idea of natural language processing (NLP) came into being in the 1950s. It is where the intellectual fields of linguistics and artificial intelligence intersect [2]. Even though the natural languages humans use have grammatical and syntactical rules, they are also steeped in nuances and can rely heavily on context to be properly interpreted. Things like tones and dialects continue to evolve and language as a whole will always be under the influence of the current generation. There are other language oddities such as sarcasm and puns that are also subject to context when being interpreted [3].

Broadly put, natural language processing consists of techniques utilized by a computer to extract meaning and value from natural language [3]. There are a multitude of problems that it can help tackle. Some of these tasks include speech recognition, part-of-speech tagging, word-sense disambiguation, named entity recognition, co-reference resolution, sentiment analysis, and even natural language generation [4].

B. Cleaning Up Text Data

When performing any kind of natural language processing, it is both recommended and common practice to clean the text being worked with and break it down into basic components in the form of individual words or groups of words. This is called tokenizing [3]. It is also very useful to remove text of any useless punctuation or other non-alphanumeric characters before tokenizing.

As a next step, small words, also called stop words, that are common like, “a,” “do,” and “it,” may be removed, as they do not contribute much in terms of meaning [3].

Following that, stemming and/or lemmatization may be performed. Stemming helps to further reduce semantic complexity by mapping words down to their original stem [5]. For example, the words “eating,” “eats,” and “eaten,” would all map to their stem, “eat.” On the other hand, lemmatization is similar to stemming, but it maps words to their root meaning instead [3]. For example, the words “is,” “am,” “was,” and “being,” would all be mapped to their lemma, “be.”

Beyond this, there are other pre-processing tasks that can be completed to make text even more workable. One such example includes using large databases to disambiguate words based on the context in which they appear. A sentence referring to a guitar as an instrument will give different value to the word “instrument” than a sentence using that word to refer to a surgical tool [3].

C. Resources and Previous Research on Clickbait Detection

Having looked at other previous works and multiple other sources, I will attempt to apply what I have learned and develop my own version of a program that can attempt to identify whether or not a title is clickbait.

II. METHODOLOGY

Due to time constraints brought on by the end of the semester and pressure from workloads in other classes, my implementation for a solution to this clickbait problem did not dive too deeply beyond what has already been attempted. That being said, even though my implementation is based on already-existing ideas, I worked to keep my code original in

the sense that it is being applied uniquely to this problem from my perspective.

As multiple sources, like IBM’s website, share, Python has quite a few tools and libraries available to assist in solving natural language processing problems. For one, the Natural Language Toolkit (NLTK) is documented as a great resource, being an open-source collection of libraries and programs for approaching natural language processing problems [4].

A. Libraries Used

For the implementation portion of this study, I made use of a variety of python libraries, while trying to make sure they are all available by default with Anaconda. Main libraries used:

- numpy
- pandas
- nltk
- sklearn
- TfidfVectorizer
- re
- string
- matplotlib
- seaborn

B. Pre-processing Data

For this study, I chose to use a data set of thirty-thousand titles [6]. This set was comprised of two columns: ‘headline’ and ‘clickbait.’ The first column simply contained a title, while the second column contained a binary value. In the ‘clickbait’ column, a value of 0 indicated a non-clickbait title while a value of 1 indicated a clickbait title. The ratio of clickbait to non-clickbait titles was roughly 50:50. According to the creator of the data set, non-clickbait titles were gathered from a variety of trustworthy sources such as WikiNews, New York Times, and The Guardian. Clickbait titles were gathered from websites such as BuzzFeed, Upworthy, and ViralNova [6].

Since the formatting of these titles was not explicitly stated, I took great lengths to pre-process it to make sure it adhered to a respectable standard. This started with removing non-alphanumeric characters. After that, I tokenized the words in each title. From there, I stemmed and lemmatized the tokens. After that, I filtered out stop words from the

titles, and finally unified the tokens back into their title strings. The results of this can be seen in this project’s accompanying Jupyter Notebook.

C. Frequency of Distribution

As a first step of analysis, I was interested to find out what words or tokens appeared most frequently in clickbait titles vs. non-clickbait titles. Fig. 1 shows the top twenty-five most common words in clickbait titles, while Fig. 2 shows the same for non-clickbait titles.

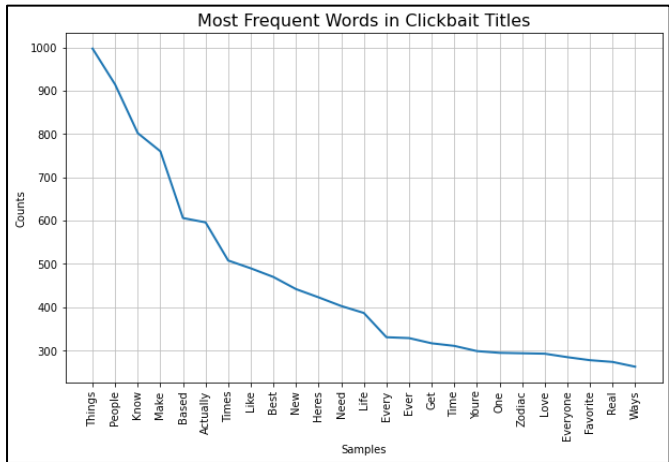


Fig. 1 Most frequent words in clickbait titles.

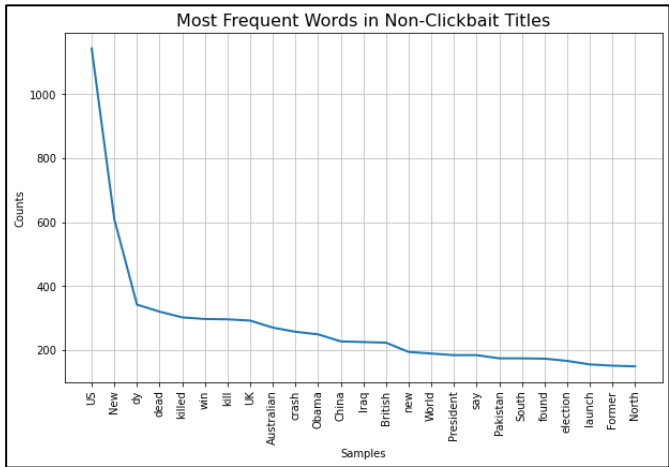


Fig. 2 Most frequent words in non-clickbait titles.

D. Splitting Data and Analysis

The data was divided by the pre-processed titles becoming an x variable, while the clickbait flag became the y variable. I split the data into a training, validation, and testing set. The data was randomized and divided by an 80:5:15 ratio.

Then, using functions from the TfidfVectorizer library, the X_train, X_valid, and X_test sets were turned into vectors so that they could be worked with.

E. Naïve Bayes

A multinomial naïve Bayes analysis resulted in a ~96% accuracy score when tested against the test set. It had a precision of ~96.5%, recall of ~96%, and produced an f1-score of 96%. Fig. 3 shows a table of this data and Fig. 4 shows a confusion matrix.

Naive Bayes accuracy: 0.960625

Classification report...

	precision	recall	f1-score	support
0	0.97	0.95	0.96	2340
1	0.96	0.97	0.96	2460
accuracy			0.96	4800
macro avg	0.96	0.96	0.96	4800
weighted avg	0.96	0.96	0.96	4800

Fig. 3 Naïve Bayes classification scores.

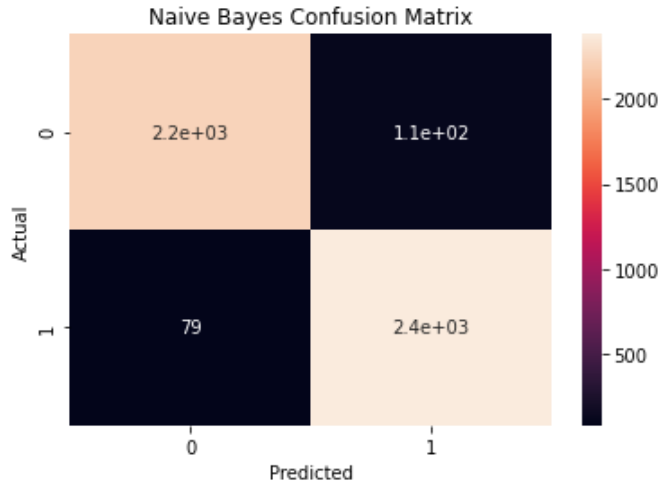


Fig. 4 Naïve Bayes confusion matrix.

F. Logistic Regression

A logistic regression analysis resulted in a ~95% accuracy score when tested against the test set. It had a precision of ~95.5%, recall of ~95.5%, and produced an f1-score of 95%. Fig. 5 shows a table of this data and Fig. 6 shows a confusion matrix.

Logistic Regression accuracy = 0.9535416666666666

Classification report...

	precision	recall	f1-score	support
0	0.94	0.97	0.95	2340
1	0.97	0.94	0.95	2460
accuracy			0.95	4800
macro avg	0.95	0.95	0.95	4800
weighted avg	0.95	0.95	0.95	4800

Fig. 5 Logistic regression classification scores.



Fig. 6 Logistic regression confusion matrix.

G. Standard Vector Machines

A standard vector machine analysis with a linear kernel resulted in a ~96% accuracy score when tested against the test set. It had a precision of ~96.7%, recall of ~95.1%, and produced an f1-score of 95%. Fig. 7 shows a table of this data and Fig. 8 shows a confusion matrix.

SVM (linear kernel) accuracy: 0.9585416666666666

SVM (linear kernel) precision: 0.9673418768085986

SVM (linear kernel) recall: 0.9512195121951219

Classification report...

	precision	recall	f1-score	support
0	0.94	0.97	0.95	2340
1	0.97	0.94	0.95	2460
accuracy			0.95	4800
macro avg	0.95	0.95	0.95	4800
weighted avg	0.95	0.95	0.95	4800

Fig. 7 SVM linear kernel classification scores.

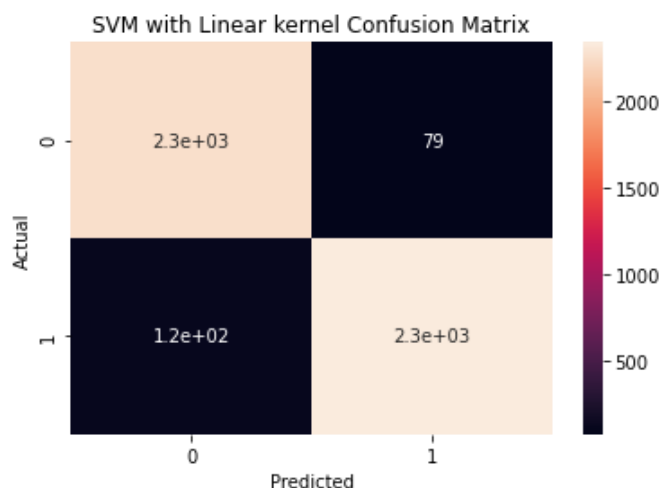


Fig. 8 SVM linear kernel confusion matrix.

To be thorough, I also performed an SVM model and analysis with sigmoid, radial basis function, and polynomial kernels as well. Information on those can be found in Fig. 9 to Fig. 14.

SVM (sigmoid kernel) accuracy: 0.9585416666666666

SVM (sigmoid kernel) precision: 0.9673418768085986

SVM (sigmoid kernel) recall: 0.9512195121951219

Classification report...

	precision	recall	f1-score	support
0	0.94	0.97	0.95	2340
1	0.97	0.94	0.95	2460
accuracy			0.95	4800
macro avg	0.95	0.95	0.95	4800
weighted avg	0.95	0.95	0.95	4800

Fig. 9 SVM sigmoid kernel classification scores.

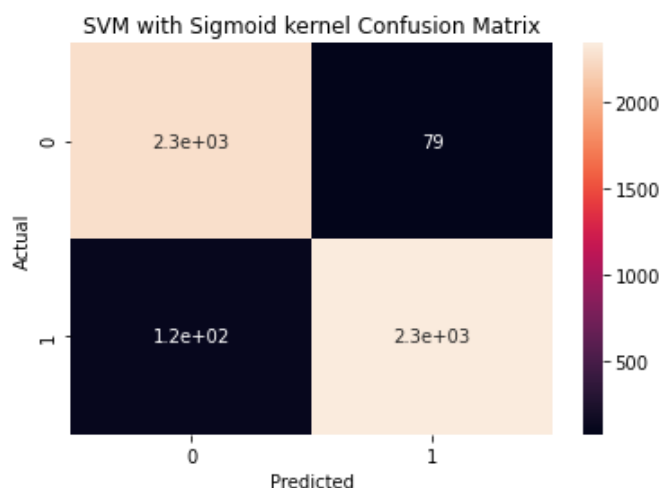


Fig. 10 SVM sigmoid kernel confusion matrix.

```
SVM (rbf kernel) accuracy: 0.9622916666666667
SVM (rbf kernel) precision: 0.9765788373065662
SVM (rbf kernel) recall: 0.9491869918699187
Classification report...
```

	precision	recall	f1-score	support
0	0.94	0.97	0.95	2340
1	0.97	0.94	0.95	2460
accuracy			0.95	4800
macro avg	0.95	0.95	0.95	4800
weighted avg	0.95	0.95	0.95	4800

Fig. 11 SVM RBF kernel classification scores.

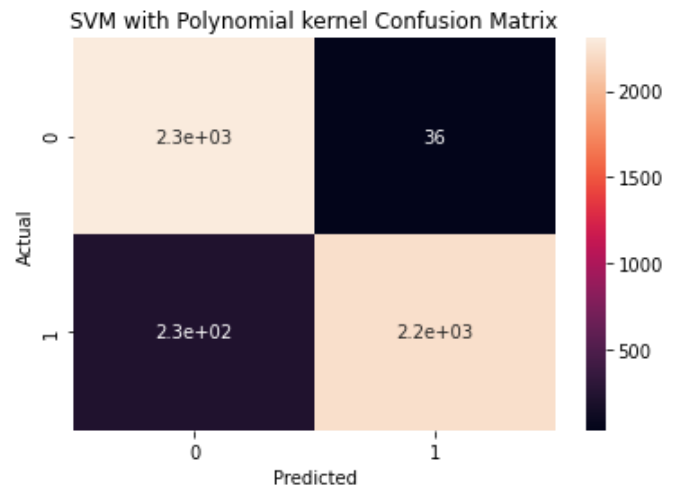


Fig. 14 SVM polynomial kernel confusion matrix.

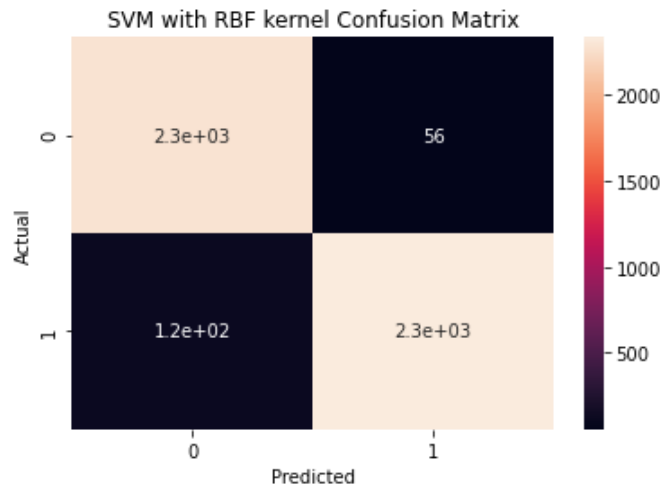


Fig. 12 SVM RBF kernel confusion matrix.

```
SVM (poly kernel) accuracy: 0.9452083333333333
SVM (poly kernel) precision: 0.9841339797267519
SVM (poly kernel) recall: 0.9077235772357723
Classification report...
```

	precision	recall	f1-score	support
0	0.94	0.97	0.95	2340
1	0.97	0.94	0.95	2460
accuracy			0.95	4800
macro avg	0.95	0.95	0.95	4800
weighted avg	0.95	0.95	0.95	4800

Fig. 13 SVM polynomial kernel classification scores.

III. DISCUSSION

A. Discussion of Results

As mentioned earlier, time and workload constraints (and poor planning) prevented me from exploring this study as much as I would have liked to. That being said, this project granted me a lot of insight and exposure to what natural language processing is, how it can be useful, and how it can be implemented, even at a basic level. The results of my study ended up being quite promising and a bit exciting. Seeing multiple models return such high accuracy scores felt very satisfying.

It was interesting to see what words showed up more frequently in each category. More vague and general lemmas like “Things,” “People,” and “Know,” were clearly more prevalent in clickbait titles. On the other hand, the non-clickbait titles saw a lot of proper nouns and informative lemmas such as “US,” “New,” and “dead.” It goes to show that an informative title starts telling a story and informing the reader before they even open the article. Clickbait titles try to hide as much information as they can so that the reader is forced to open and sift through their articles to try and find the information they are looking for.

All models produced accuracy values upward of 90%. The standard vector machine using a radial basis function kernel returned the highest accuracy value of ~96.2%. The lowest accuracy score belonged to the standard vector machine using a polynomial kernel, at ~94.5%.

In addition to the promising scores, the confusion matrices also show very positive results, showing that only a fraction of titles were mislabelled by each of the models.

B. Idea Goals for Furter Investigation

There are a lot more nuanced aspects of this problem that I did not get to fully explore with the limited time that I had. During my research, I came across other various tools such as the Gensim Python library and the word2vec algorithm [7]. Word embedding using the keras library [8] also seemed interesting, as well as bigger projects such as Google's project involving embeddings in Bi-directional Encoder Representations from Transformers (BERT) [9]. Had I more time, I feel I may have been able to test different methods of solving the problem of this project.

IV. CONCLUSION

From the results of this short study, it became apparent that this clickbait title problem has been explored quite a bit already. With that said, there is no single perfect way to approach this problem. I was able to test a few different models from naïve Bayes, to SVMs. From this, one can see that with a dataset that provides binary classification of titles as clickbait or not clickbait such as the one that I used, it is possible to quantify these titles as vectors and perform analysis that renders fairly accurate prediction models.

Next steps for a project like this could possibly include applying one or more of these models to a system that could filter out or hide online articles or videos with clickbait titles from a user. That way, they would be able to reclaim the time that they might waste sifting through the mountains of clickbait that resides on the internet.

REFERENCES

- [1] A. Chakraborty, B. Paranjape, S. Kakarla and N. Ganguly, "Stop Clickbait: Detecting and preventing clickbaits in online news media," *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 2016, pp. 9-16, doi: 10.1109/ASONAM.2016.7752207.
- [2] P. M. Nadkarni, L. Ohno-Machado, and W. W. Chapman, "Natural language processing: An introduction," *Journal of the American Medical Informatics Association*, vol. 18, no. 5, pp. 544–551, 2011.
- [3] J. M. Wyatt, G. J. Booth, and A. H. Goldman, "Natural language processing and its use in Orthopaedic Research," *Current Reviews in Musculoskeletal Medicine*, pp. 1–5, Nov. 2021.
- [4] IBM Cloud Education, "What is natural language processing?," IBM, 02-Jul-2020. [Online]. Available: <https://www.ibm.com/cloud/learn/natural-language-processing>.
- [5] J. Grimmer and B. M. Stewart, "Text as Data: The Promise and Pitfalls of Automatic Content Analysis Methods for Political Texts," *Political Analysis*, vol. 21, no. 3, pp. 267–297, 2013.
- [6] A. Anand, "Clickbait Dataset," *Kaggle*, 18-Apr-2020. [Online]. Available: <https://www.kaggle.com/amananandrai/clickbait-dataset/version/1>. [Accessed: 02-Dec-2021].
- [7] J. Brownlee, "How to Develop Word Embeddings in Python with Gensim," *Machine Learning Mastery*, 03-Sep-2020. [Online]. Available: <https://machinelearningmastery.com/develop-word-embeddings-python-gensim/>. [Accessed: Dec-2021].
- [8] T. Balodi, "Word embedding in NLP with Python Code Implementation," *Analytics Steps*, 08-Dec-2020. [Online]. Available: <https://www.analyticssteps.com/blogs/word-embedding-nlp-python-code>. [Accessed: Dec-2021].
- [9] A. N. T, "Embeddings in Bert," *OpenGenus IQ: Computing Expertise & Legacy*, 05-Oct-2021. [Online]. Available: <https://iq.opengenus.org/embeddings-in-bert/>. [Accessed: Dec-2021].